Reconstructing Attention from Hardware Up: Deep Dive into FlashAttention on AWS Neuron (NKI)

Subtitle: Why Operator Fusion and Online Softmax are the lifelines of LLM inference performance in 2026.

Introduction

The deep learning field has been experiencing a **seismic shift.** The bottleneck of Large Language Model (LLM) inference has moved from "lack of compute" to being **memory-bound.** On AWS Neuron architecture (Trainium/Inferentia2), traditional Attention implementations suffer from massiveO(N2)memory I/O overhead, making long-context sequences prohibitively expensive.

In this post, we explore how to leverage the **Neuron Kernel Interface (NKI)** to implement **FlashAttention** and dissect its mathematical soul: **Online Softmax.**

1. The Hardware Reality: NeuronCore Constraints

Writing an NKI kernel requires a "hardware-first" mindset. The **SBUF (Scratchpad Memory)** inside the NeuronCore is the source of extreme performance, but it comes with two rigid constraints:

- **The 128 Partition Limit:** The Neuron Instruction Set Architecture (ISA) requires the first dimension of any Tile (the **Partition Dimension**) to be

no larger than **128.**

- **Capacity Limits**: The SBUF is tiny (typically ~2MB) compared to the gigabytes of HBM (High Bandwidth Memory).

The Challenge: Even a "small" 512×512 attention score matrix, when processed across 128 parallel lanes (Heads), requires 64MB, which would instantly crash the SBUF. We must abandon the "load-everything" illusion and embrace a **Tile-based** strategy.

2. The Core Algorithm: The Magic of 1-Pass

Standard Attention is a **3-pass** process:

1. Read back Q*KT, compute Softmax, write back to HBM.
2. Read back Softmax weights and V, compute output, write to HBM.

FlashAttention aims for a **1-pass** approach: data is read from HBM into SBUF exactly once, processed entirely "on the cutting board," and only the final result is written back.

The Secret Sauce: Online Softmax

To compute a global Softmax while only seeing small blocks of data, we use the **Online Softmax** algorithm.

A Conceptual Analogy:

Imagine a high-jump competition with 512 students. You don't need to wait for everyone to finish before scoring. You keep a running log:

- If the 10th student sets a new record (a new **Global Maximum m**), you don't re-measure the previous 9 students.
- Instead, you use a **Rescaling Factor** — to "discount" your previous local sums, mathematically aligning them to the new global reality.

$$e^{(m_{old}-m_{new})}$$

Rescaling Factor

This **"Dynamic Re-calibration"** allows us to process tiles incrementally without ever storing the full 512 × 512 matrix in HBM.

3. NKI Kernel Workflow

In a standard NKI FlashAttention implementation, the data flow looks like this:

1. **Outer Loop**: Handle Batch×Heads. We map these tasks to the 128 P-dim lanes for massive parallelism.
2. **Inner Loop**: Tile *K* and V in chunks (e.g.,128 columns at a time).
3. **The Collision**: The **Tensor Engine** uses its systolic array to compute a 128×128 matrix multiplication in a single hardware burst.
4. **In-place Correction**: The **VPU (Vector Processing Unit)** immediately calculates the **Rescaling Factor** and updates the accumulator tile directly in SBUF.
5. **Pipelining**: Using asynchronous DMA, we pre-fetch the next K,V(128 x 128) block while the Tensor Engine is calculating the current one, achieving **Compute/IO Overlap**.

Question? *what if we have Seq=512 and Dim=128, how many operations there will be?*

4. Industry Context: vLLM and Multi-tenant Serving

Why go through the trouble of writing low-level kernels? In frameworks like **vLLM**, Multi-LoRA support is built on these exact principles.

- **Prefill Phase:** Uses fused kernels like FlashAttention to handle dense $O(N2)$ computations efficiently.
- **Multi-tenancy:** Uses **Stacked Weights** and **BGMV/SGMM kernels** to dynamically swap different LoRA adapters for different requests within a single batch.

This architecture — shared base model, fused operators, and dynamic indexing — is the bedrock of commercializing LLMs at scale in 2026.

5. Conclusion: Lessons from the Bottom

Learning NKI or CUDA is not an end in itself; the goal is to master **IO-aware programming**.

- **HBM** is the warehouse (Huge but slow).
- **SBUF** is the cutting board (Fast but small).
- **NKI Programming** is simply an "Efficient Logistics & Processing Plan."

The **Rescaling Factor** is more than just a mathematical term; it is the bridge between hardware limitations and mathematical rigor.

FAQ: The Architecture of Parallelism (Q&A)

Q: If we have Seq=512 and Dim=128, how are the "16 operations" actually executed on the NeuronCore?

*A: This is where we distinguish between **Hardware Parallelism** and **Algorithmic Tiling**. Since the NeuronCore has **128 physical Partition Lanes**, the process is broken down as follows:*

- ***Row-wise Decoupling**: The 512 tokens are split into **4 groups** (128 tokens per group). Because the Softmax normalization for each row is strictly independent, these 4 groups have no data dependency on each other.*
- ***Lane Utilization**: Each group of 128 tokens occupies the 128 hardware lanes. They undergo **4 sequential matrix operations** (scanning across the 4 tiles of K and V to cover the 512-column sequence) to complete their **Online Softmax** recalibration.*
- ***The Total Count**: We process 4 row-groups × 4 column-tiles per group = **16 total matrix operations**. While the row-groups are mathematically independent, they run sequentially in 4 batches because we have 128 physical lanes available at a time.*

Q: Does this mean we are launching 16 separate kernels?

A: No, this is a Single Fused Kernel.
In high-performance computing, the entire logic is encapsulated within one `@nki.jit` function. The 16-step tiling is handled by **loops inside the kernel.** This avoids the massive overhead of multiple kernel launches and allows intermediate data to circulate within the SBUF at terabyte-per-second speeds.

Q: Do we wait for the entire kernel to finish before writing results back to HBM?

A: Absolutely not. We use an **Asynchronous Write-back** strategy:

- **Incremental Output**: Every 4 column-scans (after completing a 128-row block), the results for those 128 tokens are fully normalized in SBUF.
- **Non-blocking DMA**: The kernel immediately triggers an asynchronous `nl.store` instruction.
- **Compute/IO Overlap**: While the DMA engine is busy moving that 128-row slice back to HBM, the Tensor Engine is already crunching the next 128-row block. This "pipelining" hides the latency of writing back to memory.

Q: Why not store the full 512x512 matrix in SBUF and write it all at once?

A: We are hitting the **Physical SBUF Ceiling**.
With a typical 2MB SBUF, it is impossible to store the 512×512 results for all 128 lanes (which would require **64MB**). By "streaming" 128-row segments back to HBM, we maintain 100% hardware occupancy while keeping our on-chip memory footprint well under the 2MB limit. We trade a few extra DMA calls for the ability to handle massive, multi-head parallelism.